



Corso Oracle SQL · Giornata 4

Manipolazione Dati e Transazioni

Una guida pratica all'uso di **INSERT**, **UPDATE**, **DELETE** e alla gestione delle transazioni in Oracle Database. Impareremo a modificare i dati in modo controllato, evitando errori critici e sfruttando **COMMIT**, **ROLLBACK** e **SAVEPOINT** per garantire l'integrità delle informazioni.



SVILUPPATORI & ANALISTI



ORACLE DATABASE



LIVELLO BASE/INTERMEDIO

Agenda della giornata

Obiettivi formativi

Nelle giornate precedenti abbiamo imparato a **interrogare** il database con SELECT, WHERE, JOIN e GROUP BY. Oggi facciamo il salto successivo:

Concetti fondamentali

Tipi di operazioni DML

Il **DML** (Data Manipulation Language) è il sottoinsieme di SQL dedicato alla manipolazione dei dati. A differenza di **SELECT**, che è un comando di sola lettura, i comandi DML **modificano lo stato** del database.



INSERT

Inserisce nuovi record in una tabella.



UPDATE

Modifica i valori di record esistenti.



DELETE

Rimuove record da una tabella.

📌 **SELECT vs DML:** **SELECT** legge i dati senza modificarli e non richiede **COMMIT**. I comandi DML, invece, alterano il contenuto del database e devono essere confermati (**COMMIT**) o annullati (**ROLLBACK**).

Comando INSERT

Inserire nuovi dati

Il comando **INSERT** aggiunge nuovi record a una tabella. È buona pratica **specificare sempre le colonne** di destinazione: il codice risulta più leggibile e resiste a eventuali modifiche della struttura della tabella.

- Indica esplicitamente le colonne coinvolte
- Rispetta i vincoli di integrità (PK, NOT NULL, CHECK)
- Usa valori coerenti con i tipi di dato

Esempio pratico

```
INSERT INTO employees
  (employee_id, first_name,
   last_name, salary)
VALUES
  (200, 'Luca', 'Test', 3000);
```

Aggiunge un nuovo dipendente alla tabella **employees** con ID 200 e stipendio 3000€.

✅ Dopo un INSERT, è sempre consigliabile verificare il risultato con una SELECT prima di eseguire il COMMIT.

Comando UPDATE & DELETE

Modificare ed eliminare dati

UPDATE

Modifica i valori di record già esistenti. La clausola **WHERE** è fondamentale: definisce **quali righe** verranno aggiornate.

```
UPDATE employees  
SET salary = salary * 1.1  
WHERE department_id = 50;
```

Aumenta del 10% lo stipendio dei dipendenti del reparto 50.

DELETE

Elimina record dalla tabella. Anche qui la clausola **WHERE** è essenziale per evitare cancellazioni massive.

```
DELETE FROM employees  
WHERE department_id = 10;
```

Cancella tutti i dipendenti del reparto 10.

 Ricorda: sia UPDATE che DELETE **senza WHERE** agiscono sull'intera tabella. Controlla sempre la clausola prima di eseguire.

⚠️ Attenzione · Rischi operativi

Errori comuni da evitare

Gli errori più pericolosi in produzione non derivano da sintassi sbagliate, ma da operazioni corrette che colpiscono troppe righe. Ecco i tre rischi principali e come proteggersi.

🚫 UPDATE senza WHERE

Modifica **tutte le righe** della tabella. Un semplice `UPDATE employees SET salary = 0;` azzerà gli stipendi di ogni dipendente.

💣 DELETE senza WHERE

Cancella l'intero contenuto della tabella. Sembra banale, ma è una delle cause più frequenti di incidenti in ambiente di produzione.

✅ Verifica con SELECT

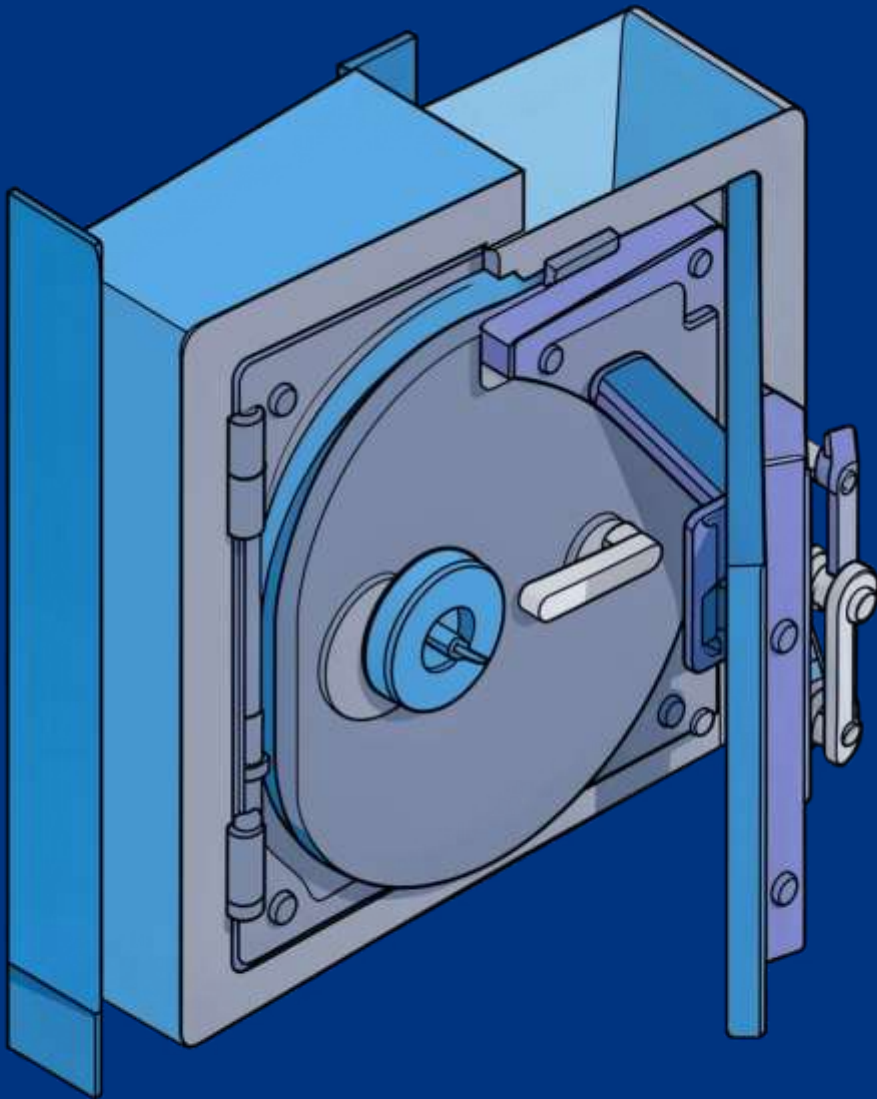
Prima di eseguire UPDATE o DELETE, esegui una **SELECT** con la stessa **WHERE** per vedere esattamente quali righe saranno impattate.

⊗ **Best practice:** lavora sempre in una transazione esplicita. In caso di errore, un ROLLBACK ti salverà la giornata.

Controllo delle modifiche

Le Transazioni

Una **transazione** è un gruppo di operazioni DML trattate come **un'unica unità logica**: o vengono applicate tutte, o nessuna. Questo garantisce la coerenza dei dati anche in caso di errori.



Gruppo di operazioni

Più INSERT, UPDATE e DELETE eseguiti in sequenza come un blocco coerente.



COMMIT

Conferma definitivamente le modifiche: i dati diventano permanenti e visibili agli altri utenti.



ROLLBACK

Annulla tutte le modifiche effettuate dopo l'ultimo COMMIT, ripristinando lo stato precedente.

Esempio pratico

Simulare un aumento e annullarlo

```
-- 1. Aumento di 1000€ per tutti
UPDATE employees
SET salary = salary + 1000;

-- 2. Verifica del risultato
SELECT salary FROM employees;

-- 3. Annulliamo tutto
ROLLBACK;
```

Cosa succede?

1. L'UPDATE modifica tutti gli stipendi **solo in sessione**.
2. La SELECT mostra i valori aggiornati, visibili solo all'utente corrente.
3. Il ROLLBACK **annulla** le modifiche: i dati tornano allo stato originale.

 Finché non esegui COMMIT, le modifiche sono **reversibili** e invisibili agli altri utenti.

Controllo granulare

SAVEPOINT: il salvataggio intermedio

Il **SAVEPOINT** crea un punto di salvataggio **all'interno** di una transazione. Permette di annullare solo una parte delle modifiche senza dover ripetere tutto da capo.

Esempio

```
UPDATE employees SET salary = 5000  
WHERE employee_id = 100;
```

```
SAVEPOINT prima_modifica;
```

```
UPDATE employees SET salary = 9999  
WHERE department_id = 50;
```

```
-- Ops, errore!  
ROLLBACK TO prima_modifica;
```

```
COMMIT;
```

Vantaggi

- **Recupero selettivo:** annulla solo le ultime operazioni
- **Flessibilità:** più savepoint nella stessa transazione
- **Sicurezza:** ideale per script complessi con più step

✓ Un ROLLBACK TO savepoint **non chiude** la transazione: puoi continuare a lavorare.

Riepilogo e laboratorio

Esercitazioni e concetti chiave

☒ Esercizi pratici

- Inserire 3 nuovi dipendenti nella tabella employees
- Aggiornare gli stipendi di un reparto specifico
- Cancellare record di test con WHERE preciso
- Utilizzare COMMIT e ROLLBACK in una transazione
- Creare un SAVEPOINT e testare il recupero parziale

🔑 Concetti chiave

DML — INSERT, UPDATE, DELETE per manipolare i dati.

WHERE — la clausola che salva la vita: sempre verificare prima.

Transazioni — COMMIT, ROLLBACK e SAVEPOINT per il controllo totale.

📄 🎯 **Obiettivo raggiunto:** ora sei in grado di modificare dati in Oracle Database in modo consapevole, sicuro e reversibile.